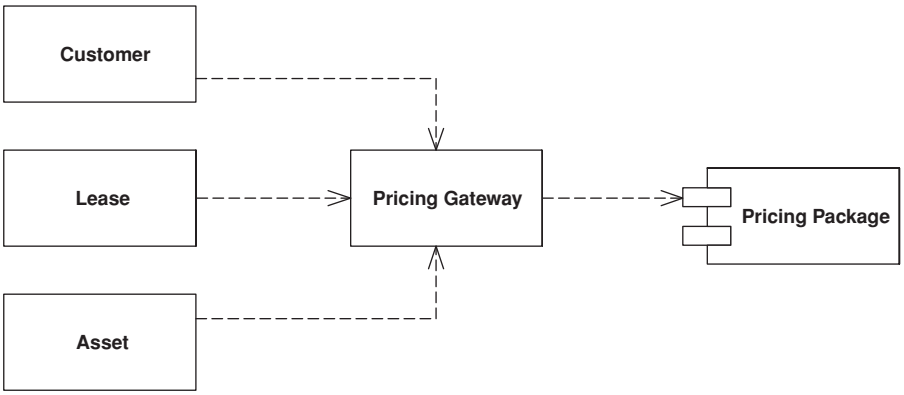
Chapter 18

Base Patterns

From the Library of Kyle Geoffrey Passarelli

Gateway

An object that encapsulates access to an external system or resource.



Interesting software rarely lives in isolation. Even the purest object-oriented system often has to deal with things that aren't objects, such as relational database tables, CICS transactions, and XML data structures.

When accessing external resources like this, you'll usually get APIs for them. However, these APIs are naturally going to be somewhat complicated because they take the nature of the resource into account. Anyone who needs to understand a resource needs to understand its API—whether JDBC and SQL for relational databases or W3C or JDOM for XML. Not only does this make the software harder to understand, it also makes it much harder to change should you shift some data from a relational database to an XML message at some point in the future.

The answer is so common that it's hardly worth stating. Wrap all the special API code into a class whose interface looks like a regular object. Other objects access the resource through this *Gateway*, which translates the simple method calls into the appropriate specialized API.

How It Works

In reality this is a very simple wrapper pattern. Take the external resource. What does the application need to do with it? Create a simple API for your usage and use the *Gateway* to translate to the external source.

From the Library of Kyle Geoffrey Passarelli

One of the key uses for a *Gateway* is as a good point at which to apply a *Service Stub (504)*. You can often alter the design of the *Gateway* to make it easier to apply a *Service Stub (504)*. Don't be afraid to do this—well placed *Service Stubs (504)* can make a system much easier to test and thus much easier to write.

Keep a *Gateway* as simple as you can. Focus on the essential roles of adapting the external service and providing a good point for stubbing. The *Gateway* should be as minimal as possible and yet able to handle these tasks. Any more complex logic should be in the *Gateway's* clients.

Often it's a good idea to use code generation to create *Gateways*. By defining the structure of the external resource, you can generate a *Gateway* class to wrap it. You might use relational metadata to create a wrapper class for a relational table, or an XML schema or DTD to generate code for a *Gateway* for XML. The resulting *Gateways* are dumb but they do the trick. Other objects can carry out more complicated manipulations.

Sometimes a good strategy is to build the *Gateway* in terms of more than one object. The obvious form is to use two objects: a back end and a front end. The back end acts as a minimal overlay to the external resource and doesn't simplify the resource's API at all. The front end then transforms the awkward API into a more convenient one for your application to use. This approach is good if the wrapping of the external service and the adaptation to your needs are reasonably complicated, because each responsibility is handled by a single class. Conversely, if the wrapping of the external service is simple, one class can handle that and any adaptation that's needed.

When to Use It

You should consider *Gateway* whenever you have an awkward interface to something that feels external. Rather than let the awkwardness spread through the whole system, use a *Gateway* to contain it. There's hardly any downside to making the *Gateway*, and the code elsewhere in the system becomes much easier to read.

Gateway usually makes a system easier to test by giving you a clear point at which to deploy *Service Stubs (504)*. Even if the external system's interface is fine, a *Gateway* is useful as a first move in applying *Service Stub (504)*.

A clear benefit of *Gateway* is that it also makes it easier for you to swap out one kind of resource for another. Any change in resources means that you only have to alter the *Gateway* class—the change doesn't ripple through the rest of the system. *Gateway* is a simple and powerful form of protected variation. In many cases reasoning about this flexibility is the focus of debate about using

From the Library of Kyle Geoffrey Passarelli

Gateway. However, don't forget that even if you don't think the resource is ever going to change, you can benefit from the simplicity and testability *Gateway* gives you.

When you have a couple of subsystems like this, another choice for decoupling them is a *Mapper (473)*. However, *Mapper (473)* is more complicated than *Gateway*. As a result, I use *Gateway* for the majority of my external resource access.

I must admit that I've struggled a fair bit with whether to make this a new pattern as opposed to referencing existing patterns such as Facade and Adapter [Gang of Four]. I decided to separate it out from these other patterns because I think there's a useful distinction to be made.

- While *Facade* simplifies a more complex API, it's usually done by the writer of the service for general use. A *Gateway* is written by the client for its particular use. In addition, a Facade always implies a different interface to what it's covering, whereas a *Gateway* may copy the wrapped facade entirely, being used for substitution or testing purposes.

- *Adapter* alters an implementation's interface to match another interface you need to work with. With *Gateway* there usually isn't an existing interface, although you might use an adapter to map an implementation to a *Gateway* interface. In this case the adapter is part of the *Gateway* implementation.

- *Mediator* usually separates multiple objects so that they don't know about each other but do know about the mediator. With a *Gateway* there are usually only two objects involved and the resource that's being wrapped doesn't know about the *Gateway*.

Example: A Gateway to a Proprietary Messaging Service (Java)

I was talking about this pattern with my colleague, Mike Rettig, and he described how he's used it to handle interfaces with Enterprise Application Integration (EAI) software. We decided that this would be a good inspiration for a *Gateway* example.

To keep things at the usual level of ludicrous simplicity, we'll build a gateway to an interface that just sends a message using the message service. The interface is just a single method.

```
int send(String messageType, Object[] args);
```

The first argument is a string indicating the type of the message; the second is the arguments of the message. The messaging system allows you to send any

From the Library of Kyle Geoffrey Passarelli

kind of message, so it needs a generic interface like this. When you configure the message system you specify the types of message the system will send and the number and types of arguments for them. Thus, we might configure the confirm message with the string "CNFRM" and have arguments for an ID number as a string, an integer amount, and a string for the ticker code. The messaging system checks the types of the arguments for us and generates an error if we send a wrong message or the right message with the wrong arguments.

This is laudable, and necessary, flexibility, but the generic interface is awkward to use because it isn't explicit. You can't tell by looking at the interface what the legal message types are or what arguments are needed for a certain message type. What we need instead is an interface with methods like this:

```
public void sendConfirmation(String orderID, int amount, String symbol);
```

That way if we want a domain object to send a message, it can do so like this:

```
class Order...

   public void confirm() {
      if (isValid()) Environment.getMessageGateway().sendConfirmation(id, amount, symbol);
   }
```

Here the name of the method tells us what message we're sending, and the arguments are typed and given names. This is a much easier method to call than the generic method. It's the gateway's role to make a more convenient interface. It does mean, though, that every time we add or change a message type in the messaging system we need to change the gateway class, but we would have to change the calling code anyway. At least this way the compiler can help us find clients and check for errors.

There's another problem. When we get an error with this interface it tells us by giving us a return error code. Zero indicates success; anything else indicates failure, and different numbers indicate different errors. This is a natural way for a C programmer to work, but it isn't the way Java does things. In Java you throw an exception to indicate an error, so the *Gateway's* methods should throw exceptions rather than return error codes.

The full range of possible errors is something that we'll naturally ignore. I'll focus on just two: sending a message with an unknown message type and sending a message where one of the arguments is null. The return codes are defined in the messaging system's interface.


```
public static final int NULL_PARAMETER = -1;
public static final int UNKNOWN_MESSAGE_TYPE = -2;
public static final int SUCCESS = 0;
```

From the Library of Kyle Geoffrey Passarelli

The two errors have a significant difference. The unknown message type error indicates an error in the gateway class; since any client is only calling a fully explicit method, clients should never generate this error. They might pass in a null, however, and thus see the null parameter error. This error isn't a checked exception since it indicates a programmer error—not something that you would write a specific handler for. The gateway could actually check for nulls itself, but if the messaging system is going to raise the same error it probably isn't worth it.

For these reasons the gateway has to both translate from the explicit interface to the generic interface and translate the return codes into exceptions.

```
class MessageGateway...

   protected static final String CONFIRM = "CNFRM";
   private MessageSender sender;
   public void sendConfirmation(String orderID, int amount, String symbol) {
      Object[] args = new Object[]{orderID, new Integer(amount), symbol};
      send(CONFIRM, args);
   }
   private void send(String msg, Object[] args) {
      int returnCode = doSend(msg, args);
      if (returnCode == MessageSender.NULL_PARAMETER)
         throw new NullPointerException("Null Parameter passed for msg type: " + msg);
      if (returnCode != MessageSender.SUCCESS)
         throw new IllegalStateException(
               "Unexpected error from messaging system #:" + returnCode);
   }
   protected int doSend(String msg, Object[] args) {
      Assert.notNull(sender);
      return sender.send(msg, args);
   }
```

So far, it's hard to see the point of the `doSend` method, but it's there for another key role for a gateway—testing. We can test objects that use the gateway without the message-sending service being present. To do this we need to create a *Service Stub (504)*. In this case the gateway stub is a subclass of the real gateway and overrides `doSend`.

```
class MessageGatewayStub...

   protected int doSend(String messageType, Object[] args) {
      int returnCode = isMessageValid(messageType, args);
      if (returnCode == MessageSender.SUCCESS) {
         messagesSent++;
      }
      return returnCode;
   }
   private int isMessageValid(String messageType, Object[] args) {
      if (shouldFailAllMessages) return -999;
```


From the Library of Kyle Geoffrey Passarelli

```
    if (!legalMessageTypes().contains(messageType))
       return MessageSender.UNKNOWN_MESSAGE_TYPE;
    for (int i = 0; i < args.length; i++) {
       Object arg = args[i];
       if (arg == null) {
          return MessageSender.NULL_PARAMETER;
       }
    }
    return MessageSender.SUCCESS;
 }
 public static List legalMessageTypes() {
    List result = new ArrayList();
    result.add(CONFIRM);
    return result;
 }
 private boolean shouldFailAllMessages = false;
 public void failAllMessages() {
    shouldFailAllMessages = true;
 }
 public int getNumberOfMessagesSent() {
    return messagesSent;
 }
```

Capturing the number of messages sent is a simple way of helping us test that the gateway works correctly with tests like these.

```
class GatewayTester...

 public void testSendNullArg() {
    try {
       gate().sendConfirmation(null, 5, "US");
       fail("Didn't detect null argument");
    } catch (NullPointerException expected) {
    }
    assertEquals(0, gate().getNumberOfMessagesSent());
 }
 private MessageGatewayStub gate() {
    return (MessageGatewayStub) Environment.getMessageGateway();
 }
 protected void setUp() throws Exception {
    Environment.testInit();
 }
```

You usually set up the *Gateway* so that classes can find it from a well-known place. Here I've used a static environment interface. You can switch between the real service and the stub at configuration time by using a *Plugin (499)*, or you can have the test setup routines initialize the environment to use the *Service Stub (504)*.

In this case I've used a subclass of the gateway to stub the messaging service. Another route is to subclass (or reimplement) the service itself. For testing you

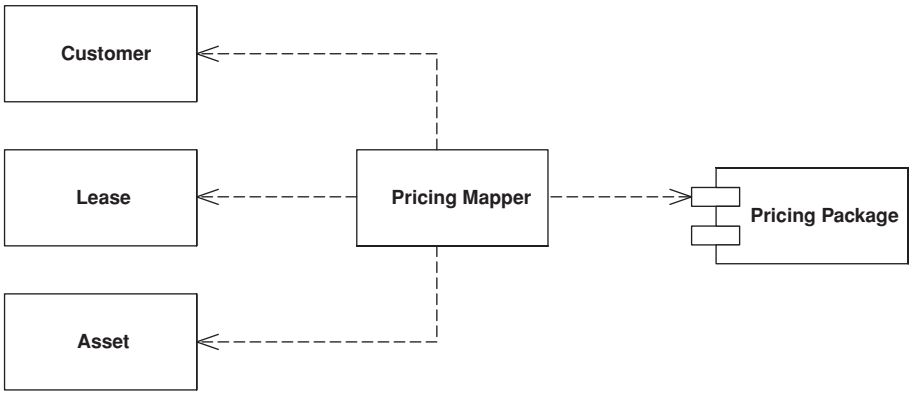
From the Library of Kyle Geoffrey Passarelli

connect the gateway to the sending *Service Stub (504);* it works if reimplementation of the service isn't too difficult. You always have the choice of stubbing the service or stubbing the gateway. In some cases it's even useful to stub both, using the stubbed gateway for testing clients of the gateway and the stubbed service to test the gateway itself.

From the Library of Kyle Geoffrey Passarelli

Mapper

An object that sets up a communication between two independent objects.



Sometimes you need to set up communications between two subsystems that still need to stay ignorant of each other. This may be because you can't modify them or you can but you don't want to create dependencies between the two or even between them and the isolating element.

How It Works

A mapper is an insulating layer between subsystems. It controls the details of the communication between them without either subsystem being aware of it.

A mapper often shuffles data from one layer to another. Once activated for this shuffling, it's fairly easy to see how it works. The complicated part of using a mapper is deciding how to invoke it, since it can't be directly invoked by either of the subsystems that it's mapping between. Sometimes a third subsystem drives the mapping and invokes the mapper as well. An alternative is to make the mapper an observer [Gang of Four] of one or the other subsystem. That way it can be invoked by listening to events in one of them.

How a mapper works depends on the kind of layers it's mapping. The most common case of a mapping layer that we run into is in a *Data Mapper (165)*, so look there for more details on how a *Mapper* is used.

From the Library of Kyle Geoffrey Passarelli

When to Use It

Essentially a *Mapper* decouples different parts of a system. When you want to do this you have a choice between *Mapper* and *Gateway (466)*. *Gateway (466)* is by far the most common choice because it's much simpler to use a *Gateway (466)* than a *Mapper* both in writing the code and in using it later.

As a result you should only use a *Mapper* when you need to ensure that neither subsystem has a dependency on this interaction. The only time this is really important is when the interaction between the subsystems is particularly complicated and somewhat independent to the main purpose of both subsystems. Thus, in enterprise applications we mostly find *Mapper* used for interactions with a database, as in *Data Mapper (165)*.

Mapper is similar to Mediator [Gang of Four] in that it's used to separate different elements. However, the objects that use a mediator are aware of it, even if they aren't aware of each other; the objects that a *Mapper* separates aren't even aware of the mapper.

From the Library of Kyle Geoffrey Passarelli

Layer Supertype

A type that acts as the supertype for all types in its layer.

It's not uncommon for all the objects in a layer to have methods you don't want to have duplicated throughout the system. You can move all of this behavior into a common *Layer Supertype*.

How It Works

Layer Supertype is a simple idea that leads to a very short pattern. All you need is a superclass for all the objects in a layer—for example, a Domain Object superclass for all the domain objects in a *Domain Model (116)*. Common features, such as the storage and handling of *Identity Fields (216)*, can go there. Similarly all *Data Mappers (165)* in the mapping layer can have a superclass that relies on the fact that all domain objects have a common superclass.

If you have more than one kind of object in a layer, it's useful to have more than one *Layer Supertype*.

When to Use It

Use *Layer Supertype* when you have common features from all objects in a layer. I often do this automatically because I make a lot of use of common features.

Example: Domain Object (Java)

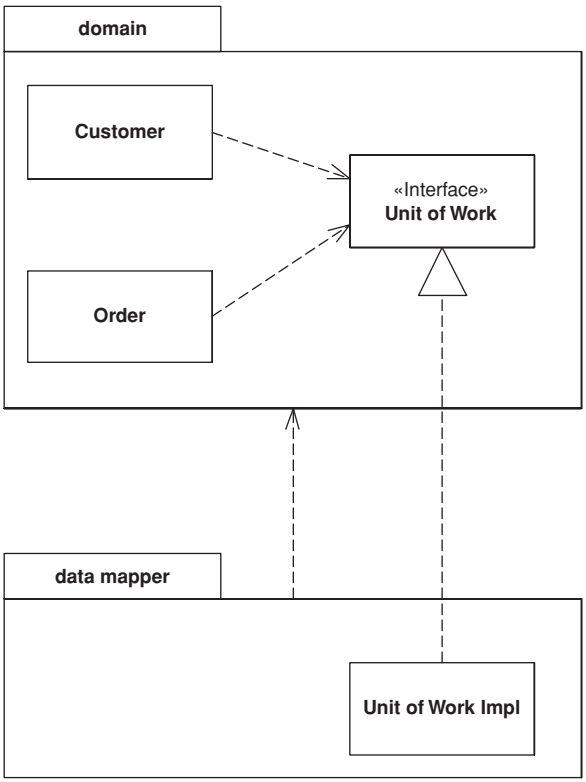
Domain objects can have a common superclass for ID handling.

```
class DomainObject...

   private Long ID;
   public Long getID() {
      return ID;
   }
   public void setID(Long ID) {
      Assert.notNull("Cannot set a null ID", ID);
      this.ID = ID;
   }
   public DomainObject(Long ID) {
      this.ID = ID;
   }
   public DomainObject() {
   }
```


From the Library of Kyle Geoffrey Passarelli

Separated Interface

Defines an interface in a separate package from its implementation.



As you develop a system, you can improve the quality of its design by reducing the coupling between the system's parts. A good way to do this is to group the classes into packages and control the dependencies between them. You can then follow rules about how classes in one package can call classes in another—for example, one that says that classes in the domain layer may not call classes in the presentation package.

However, you might need to invoke methods that contradict the general dependency structure. If so, use *Separated Interface* to define an interface in one package but implement it in another. This way a client that needs the depen-

From the Library of Kyle Geoffrey Passarelli

dency to the interface can be completely unaware of the implementation. The *Separated Interface* provides a good plug point for *Gateway (466)*.

How It Works

This pattern is very simple to employ. Essentially it takes advantage of the fact that an implementation has a dependency to its interface but not vice versa. This means you can put the interface and the implementation in separate packages and the implementation package has a dependency to the interface package. Other packages can depend on the interface package without depending on the implementation package.

Of course, the software won't work at runtime without some implementation of the interface. This can be either at compile time using a separate package that ties the two together or at configuration time using *Plugin (499)*.

You can place the interface in the client's package (as in the sketch) or in a third package (Figure 18.1). If there's only one client for the implementation, or all the clients are in the same package, then you might as well put the interface in with the client. A good way of thinking about this is that the developers of the client package are responsible for defining the interface. Essentially the client package indicates that it will work with any other package that implements the interface it defines. If you have multiple client packages, a third interface package is better. It's also better if you want to show that the interface definition isn't

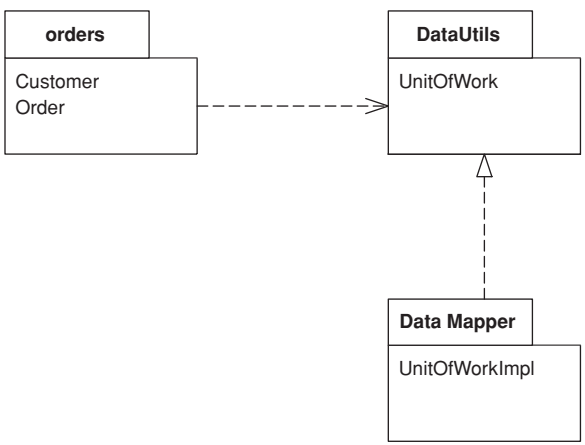


Figure 18.1 *Placing the* Separated Interface *in a third package.*

From the Library of Kyle Geoffrey Passarelli

the responsibility of the client package developers. This would be the case if the developers of the implementation were responsible for it.

You have to consider what language feature to use for the interface. For languages that have an interface construct, such as Java and C#, the interface keyword is the obvious choice. However, it may not be the best. An abstract class can make a good interface because you can have common, but optional, implementation behavior in it.

One of the awkward things about separate interfaces is how to instantiate the implementation. It usually requires knowledge of the implementation class. The common approach is to use a separate factory object, where again there is a *Separated Interface* for the factory. You still have to bind an implementation to the factory, and *Plugin (499)* is a good way to do this. Not only does it mean there is no dependency, but it also defers the decision about implementation class to configuration time.

If you don't want to go all the way to *Plugin (499)*, a simpler alternative is to let yet another package that knows both the interface and the implementation instantiate the right objects at application startup. Any objects that use *Separated Interface* can either themselves be instantiated or have factories instantiated at startup.

When to Use It

You use *Separated Interface* when you need to break a dependency between two parts of the system. Here are some examples:

- You've built some abstract code for common cases into a framework package that needs to call some particular application code.
- You have some code in one layer that needs to call code in another layer that it shouldn't see, such as domain code calling a *Data Mapper (165)*.
- You need to call functions developed by another development group but don't want a dependency into their APIs.

I come across many developers who have separate interfaces for every class they write. I think this is excessive, especially for application development. Keeping separate interfaces and implementations is extra work, especially since you often need factory classes (with interfaces and implementations) as well. For applications I recommend using a separate interface only if you want to break a dependency or you want to have multiple independent implementations. If you put the interface and implementation together and need to separate them later, this is a simple refactoring that can be delayed until you need to do it.

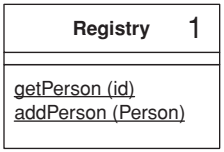
From the Library of Kyle Geoffrey Passarelli

There is a degree to where the determined management of dependencies in this way can get a little silly. Having only a dependency to create an object, and using the interface ever after, is usually enough. The trouble comes when you want to enforce dependency rules, such as by doing a dependency check at build time. Then all dependencies have to be removed. For a smaller system enforcing dependency rules is less of an issue, but for bigger systems it's a very worthwhile discipline.

From the Library of Kyle Geoffrey Passarelli

Registry

A well-known object that other objects can use to find common objects and services.



When you want to find an object you usually start with another object that has an association to it, and use the association to navigate to it. Thus, if you want to find all the orders for a customer, you start with the customer object and use a method on it to get the orders. However, in some cases you won't have an appropriate object to start with. You may know the customer's ID number but not have a reference. In this case you need some kind of lookup method—a finder—but the question remains: How do you get to the finder?

A *Registry* is essentially a global object, or at least it looks like one—even if it isn't as global as it may appear.

How It Works

As with any object, you have to think about the design of a *Registry* in terms of interface and implementation. And like many objects, the two are quite different, although people often make the mistake of thinking they should be the same.

The first thing to think of is the interface, and for *Registries* my preferred interface is static methods. A static method on a class is easy to find anywhere in an application. Furthermore, you can encapsulate any logic you like within the static method, including delegation to other methods, either static or instance.

However, just because your methods are static doesn't mean that your data should be in static fields. Indeed, I almost never use static fields unless they're constants.

Before you decide on how to hold your data, think about the data's scope. The data for a *Registry* can vary with different execution contexts. Some of it is global across an entire process; some, global across a thread; and some, global

From the Library of Kyle Geoffrey Passarelli

across a session. Different scopes call for different implementations, but they don't call for different interfaces. The application programmer doesn't have to know whether a call to a static method yields process-scoped or thread-scoped data. You can have different *Registries* for different scopes, but you can also have a single *Registry* in which different methods are at different scopes.

If your data is common to a whole process, a static field is an option. However, I rarely use static mutable fields because they don't allow for substitution. It can be extremely useful to be able to substitute a *Registry* for a particular purpose, especially for testing (*Plugin (499)* is a good way to do this).

For a process-scoped *Registry,* then, the usual option is a singleton [Gang of Four]. The *Registry* class contains a single static field that holds a *Registry* instance. When people use a singleton they often make its caller explicitly access the underlying data (`Registry.soleInstance.getFoo()`), but I prefer a static method that hides the singleton object from me (`Registry.getFoo()`). This works particularly well since C-based languages allow static methods to access private instance data.

Singletons are widely used in single-threaded applications, but can be a problem for multi-threaded applications. This is because it's too easy for multiple threads to manipulate the same object in unpredictable ways. You may be able to solve this with synchronization, but the difficulty of writing the synchronization code is likely to drive you into an insane asylum before you get all the bugs out. For that reason I don't recommend using a singleton for mutable data in a multi-threaded environment. It does work well for immutable data, since anything that can't change isn't going to run into thread clash problems. Thus, something like a list of all states in the United States makes a good candidate for a process-scoped *Registry.* Such data can be loaded when a process starts up and never need changing, or it may be updated rarely with some kind of process interrupt.

A common kind of *Registry* data is thread scoped. A good example is a database connection. In this case many environments give you some form of thread-specific storage, such as Java's thread local. Another technique is a dictionary keyed by thread whose value is an appropriate data object. A request for a connection results in a lookup in that dictionary by the current thread.

The important thing to remember about thread-scoped data is that it looks no different from process-scoped data. I can still use a method such as `Registry.getDbConnection()`, which is the same form when I'm accessing process-scoped data.

A dictionary lookup is also a technique you can use for session-scoped data. Here you need a session ID, but it can be put into a thread-scoped registry when a request begins. Any subsequent accesses for session data can look up the data

From the Library of Kyle Geoffrey Passarelli

in a map that's keyed by session using the session ID that's held in thread-specific storage.

If you're using a thread-scoped *Registry* with static methods, you may run into a performance issue with multiple threads going through them. In that case direct access to the thread's instance will avoid the bottleneck.

Some applications may have a single *Registry*; some may have several. *Registries* are usually divided up by system layer or by execution context. My preference is to divide them up by how they are used, rather than implementation.

When to Use It

Despite the encapsulation of a method, a *Registry* is still global data and as such is something I'm uncomfortable using. I almost always see some form of *Registry* in an application, but I always try to access objects through regular inter-object references instead. Basically, you should only use a *Registry* as a last resort.

There are alternatives to using a *Registry*. One is to pass around any widely needed data in parameters. The problem with this is that parameters are added to method calls where they aren't needed by the called method but only by some other method that's called several layers deep in the call tree. Passing a parameter around when it's not needed 90 percent of the time is what leads me to use a *Registry* instead.

Another alternative I've seen to a *Registry* is to add a reference to the common data to objects when they're created. Although this leads to an extra parameter in a constructor, at least it's only used by that constructor. It's still more trouble than it's often worth, but if you have data that's only used by a subset of classes, this technique allows you to restrict things that way.

One of the problems with a *Registry* is that it has to be modified every time you add a new piece of data. This is why some people prefer to use a map as their holder of global data. I prefer the explicit class because it keeps the methods explicit, so there's no confusion about what key you use to find something. With an explicit class you can just look at the source code or generated documentation to see what's available. With a map you have to find places in the system where data is read or written to the map to find out what key is used or to rely on documentation that quickly becomes stale. An explicit class also allows you to keep type safety in a statically typed language as well as to encapsulate the structure of the *Registry* so that you can refactor it as the system grows. A bare map also is unencapsulated, which makes it harder to hide the implementation. This is particularly awkward if you have to change the data's execution scope.

From the Library of Kyle Geoffrey Passarelli

So there are times when it's right to use a *Registry*, but remember that any global data is always guilty until proven innocent.

Example: A Singleton Registry (Java)

Consider an application that reads data from a database and then munges on it to turn it into information. Well, imagine a fairly simple system that uses *Row Data Gateways (152)* for data access. This system has finder objects to encapsulate the database queries. The finders are best made as instances because that way we can substitute them to make a *Service Stub (504)* for testing. We need a place to put them; a *Registry* is the obvious choice.

A singleton registry is a very simple example of the Singleton pattern [Gang of Four]. You have a static variable for the single instance.

```
class Registry...

   private static Registry getInstance() {
      return soleInstance;
   }
   private static Registry soleInstance = new Registry();
```

Everything that's stored on the registry is stored on the instance.

```
class Registry...

   protected PersonFinder personFinder = new PersonFinder();
```

To make access easier, however, I make the public methods static.

```
class Registry...

   public static PersonFinder personFinder() {
      return getInstance().personFinder;
   }
```

I can reinitialize the registry simply by creating a new sole instance.

```
class Registry...

   public static void initialize() {
      soleInstance = new Registry();
   }
```

If I want to use *Service Stubs (504)* for testing, I use a subclass instead.

```
class RegistryStub extends Registry...

   public RegistryStub() {
      personFinder = new PersonFinderStub();
   }
```


From the Library of Kyle Geoffrey Passarelli

The finder *Service Stub (504)* just returns hardcoded instances of the person *Row Data Gateway (152)*.

```
class PersonFinderStub...

   public Person find(long id) {
      if (id == 1) {
         return new Person("Fowler", "Martin", 10);
      }
      throw new IllegalArgumentException("Can't find id: " + String.valueOf(id));
   }
```

I put a method on the registry to initialize it in stub mode, but by keeping all the stub behavior in the subclass I can separate all the code required for testing.

```
class Registry...

   public static void initializeStub() {
      soleInstance = new RegistryStub();
   }
```

Example: Thread-Safe *Registry* (Java)

Matt Foemmel and Martin Fowler

The simple example above won't work for a multi-threaded application where different threads need their own registry. Java provides Thread Specific Storage variables [Schmidt] that are local to a thread, helpfully called thread local variables. You can use them to create a registry that's unique for a thread.

```
class ThreadLocalRegistry...

   private static ThreadLocal instances = new ThreadLocal();
   public static ThreadLocalRegistry getInstance() {
      return (ThreadLocalRegistry) instances.get();
   }
```

The *Registry* needs to be set up with methods for acquiring and releasing it. You typically do this on a transaction or session call boundary.


```
class ThreadLocalRegistry...

   public static void begin() {
      Assert.isTrue(instances.get() == null);
      instances.set(new ThreadLocalRegistry());
   }
   public static void end() {
      Assert.notNull(getInstance());
      instances.set(null);
   }
```

From the Library of Kyle Geoffrey Passarelli

You can then store person finders as before.

```
class ThreadLocalRegistry...

   private PersonFinder personFinder = new PersonFinder();;
   public static PersonFinder personFinder() {
      return getInstance().personFinder;
   }
```

Calls from the outside wrap their use of a registry in the begin and end methods.

```
      try {
         ThreadLocalRegistry.begin();
         PersonFinder f1 = ThreadLocalRegistry.personFinder();
         Person martin = Registry.personFinder().find(1);
         assertEquals("Fowler", martin.getLastName());
      } finally {ThreadLocalRegistry.end();
      }
```


From the Library of Kyle Geoffrey Passarelli

Value Object

A small simple object, like money or a date range, whose equality isn't based on identity.

With object systems of various kinds, I've found it useful to distinguish between reference objects and *Value Objects*. Of the two a *Value Object* is usually the smaller; it's similar to the primitive types present in many languages that aren't purely object-oriented.

How It Works

Defining the difference between a reference object and *Value Object* can be a tricky thing. In a broad sense we like to think that *Value Objects* are small objects, such as a money object or a date, while reference objects are large, such as an order or a customer. Such a definition is handy but annoyingly informal.

The key difference between reference and value objects lies in how they deal with equality. A reference object uses identity as the basis for equality—maybe the identity within the programming system, such as the built-in identity of OO programming languages, or maybe some kind of ID number, such as the primary key in a relational database. A *Value Object* bases its notion of equality on field values within the class. Thus, two date objects may be the same if their day, month, and year values are the same.

This difference manifests itself in how you deal with them. Since *Value Objects* are small and easily created, they're often passed around by value instead of by reference. You don't really care about how many March 18, 2001, objects there are in your system. Nor do you care if two objects share the same physical date object or whether they have different yet equal copies.

Most languages have no special facility for value objects. For value objects to work properly in these cases it's a very good idea to make them immutable—that is, once created none of their fields change. The reason for this is to avoid aliasing bugs. An aliasing bug occurs when two objects share the same value object and one of the owners changes the values in it. Thus, if Martin has a hire date of March 18 and we know that Cindy was hired on the same day, we may set Cindy's hire date to be the same as Martin's. If Martin then changes the month in his hire date to May, Cindy's hire date changes too. Whether it's correct or not, it isn't what people expect. Usually with small values like this people expect to change a hire date by replacing the existing date object with a new one. Making *Value Objects* immutable fulfills that expectation.

From the Library of Kyle Geoffrey Passarelli

Value Objects shouldn't be persisted as complete records. Instead use *Embedded Value (268)* or *Serialized LOB (272)*. Since *Value Objects* are small, *Embedded Value (268)* is usually the best choice because it also allows SQL querying using the data in a *Value Object*.

If you're doing a lot of binary serializing, you may find that optimizing the serialization of *Value Objects* can improve performance, particularly in languages like Java that don't treat for *Value Objects* in a special way.

For an example of a *Value Object* see *Money (488)*.

.NET Implementation

.NET has a first-class treatment of *Value Object*. In C# an object is marked as a *Value Object* by declaring it as a struct instead as a class. The environment then treats it with value semantics.

When to Use It

Treat something as a *Value Object* when you're basing equality on something other than an identity. It's worth considering this for any small object that's easy to create.

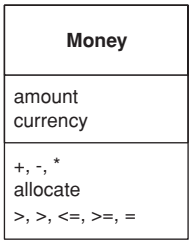
Name Collisions I've seen the term *Value Object* used for this pattern for quite some time. Sadly recently I've seen the J2EE community [Alur et al.] use the term "value object" to mean *Data Transfer Object (401)*, which has caused a storm in the teacup of the patterns community. This is just one of those clashes over names that happen all the time in this business. Recently [Alur et al.] decided to use the term *transfer object* instead.

I continue to use *Value Object* in this way in this text. If nothing else it allows me to be consistent with my previous writings!

From the Library of Kyle Geoffrey Passarelli

Money

Represents a monetary value.



A large proportion of the computers in this world manipulate money, so it's always puzzled me that money isn't actually a first class data type in any mainstream programming language. The lack of a type causes problems, the most obvious surrounding currencies. If all your calculations are done in a single currency, this isn't a huge problem, but once you involve multiple currencies you want to avoid adding your dollars to your yen without taking the currency differences into account. The more subtle problem is with rounding. Monetary calculations are often rounded to the smallest currency unit. When you do this it's easy to lose pennies (or your local equivalent) because of rounding errors.

The good thing about object-oriented programming is that you can fix these problems by creating a Money class that handles them. Of course, it's still surprising that none of the mainstream base class libraries actually do this.

How It Works

The basic idea is to have a Money class with fields for the numeric amount and the currency. You can store the amount as either an integral type or a fixed decimal type. The decimal type is easier for some manipulations, the integral for others. You should absolutely avoid any kind of floating point type, as that will introduce the kind of rounding problems that *Money* is intended to avoid. Most of the time people want monetary values rounded to the smallest complete unit, such as cents in the dollar. However, there are times when fractional units are needed. It's important to make it clear what kind of money you're working with, especially in an application that uses both kinds. It makes sense to have different types for the two cases as they behave quite differently under arithmetic.

From the Library of Kyle Geoffrey Passarelli

Money is a *Value Object (486)*, so it should have its equality and hash code operations overridden to be based on the currency and amount.

Money needs arithmetic operations so that you can use money objects as easily as you use numbers. But arithmetic operations for money have some important differences to money operations in numbers. Most obviously, any addition or subtraction needs to be currency aware so you can react if you try to add together monies of different currencies. The simplest, and most common, response is to treat the adding together of disparate currencies as an error. In some more sophisticated situations you can use Ward Cunningham's idea of a money bag. This is an object that contains monies of multiple currencies together in one object. This object can then participate in calculations just like any money object. It can also be valued into a currency.

Multiplication and division end up being more complicated due to rounding problems. When you multiply money you do it with a scalar. If you want to add 5% tax to a bill you multiply by 0.05, so you see multiplication by regular numeric types.

The awkward complication comes with rounding, particularly when allocating money between different places. Here's Matt Foemmel's simple conundrum. Suppose I have a business rule that says that I have to allocate the whole amount of a sum of money to two accounts: 70% to one and 30% to another. I have 5 cents to allocate. If I do the math I end up with 3.5 cents and 1.5 cents. Whichever way I round these I get into trouble. If I do the usual rounding to nearest then 1.5 becomes 2 and 3.5 becomes 4. So I end up gaining a penny. Rounding down gives me 4 cents and rounding up gives me 6 cents. There's no general rounding scheme I can apply to both that will avoid losing or gaining a penny.

I've seen various solutions to this problem.

- Perhaps the most common is to ignore it—after all, it's only a penny here and there. However this tends to make accountants understandably nervous.

- When allocating you always do the last allocation by subtracting from what you've allocated so far. This avoids losing pennies, but you can get a cumulative amount of pennies on the last allocation.

- Allow users of a Money class to declare the rounding scheme when they call the method. This permits a programmer to say that the 70% case rounds up and the 30% rounds down. Things can get complicated when you allocate across ten accounts instead of two. You also have to remember to round. To encourage people to remember I've seen some Money classes force a rounding parameter into the multiply operation. Not only does this force the programmer to think about what rounding she needs, it also might remind her of the tests to write. However, it gets messy if you have a lot of tax calculations that all round the same way.

From the Library of Kyle Geoffrey Passarelli

• My favorite solution: have an allocator function on the money. The parameter to the allocator is a list of numbers, representing the ratio to be allocated (it would look something like `aMoney.allocate([7,3])`). The allocator returns a list of monies, guaranteeing that no pennies get dropped by scattering them across the allocated monies in a way that looks pseudo-random from the outside. The allocator has faults: You have to remember to use it and any precise rules about where the pennies go are difficult to enforce.

The fundamental issue here is between using multiplication to determine proportional charge (such as a tax) and using it to allocate a sum of money across multiple places. Multiplication works well for the former, but an allocator works better for the latter. The important thing is to consider your intent in using multiplication or division on a monetary value.

You may want to convert from one currency to another with a method like `aMoney.convertTo(Currency.DOLLARS)`. The obvious way to do this is to look up an exchange rate and multiply by it. While this works in many situations, there are cases where it doesn't—again due to rounding. The conversion rules between the fixed euro currencies had specific roundings applied that made simple multiplication unworkable. Thus, it's wise to have a convertor object to encapsulate the algorithm.

Comparison operations allow you to sort monies. Like the addition operation, conversions need to be currency aware. You can either choose to throw an exception if you compare different currencies or do a conversion.

A *Money* can encapsulate the printing behavior. This makes it much easier to provide good display on user interfaces and reports. A Money class can also parse a string to provide a currency-aware input mechanism, which again is very useful for the user interface. This is where your platform's libraries can provide help. Increasingly platforms provide globalization support with specific number formatters for particular countries.

Storing a *Money* in a database always raises an issue, since databases also don't seem to understand that money is important (although their vendors do.) The obvious route to take is to use *Embedded Value (268)*, which results in storing a currency for every money. That can be overkill when, for instance, an account may have all its entries be in pounds. In this case you may store the currency on the account and alter the database mapping to pull the account's currency whenever you load entries.

When to Use It

I use *Money* for pretty much all numeric calculation in object-oriented environments. The primary reason is to encapsulate the handling of rounding behavior,

From the Library of Kyle Geoffrey Passarelli

which helps reduce the problems of rounding errors. Another reason to use *Money* is to make multi-currency work much easier. The most common objection to *Money* is performance, although I've only rarely heard of cases where it makes any noticeable difference, and even then the encapsulation often makes tuning easier.

Example: A Money Class (Java)

by Matt Foemmel and Martin Fowler

The first decision is what data type to use for the amount. If anyone needs convincing that a floating point number is a bad idea, ask them to run this code.

```
double val = 0.00;
for (int i = 0; i < 10; i++) val += 0.10;
System.out.println(val == 1.00);
```

With floats safely disposed of, the choice lies between fixed-point decimals and integers, which in Java boils down to `BigDecimal`, `BigInteger` and `long`. Using an integral value actually makes the internal math easier, and if we use `long` we can use primitives and thus have readable math expressions.

```
class Money...

  private long amount;
  private Currency currency;
```

I'm using an integral amount, that is, the amount of the smallest base unit, which I refer to as cents in the code because it's as good a name as any. With a long we get an overflow error if the number gets too big. If you give us $92,233,720,368,547,758.09 we'll write you a version that uses `BigInteger`.

It's useful to provide constructors from various numeric types.

```
  public Money(double amount, Currency currency) {
     this.currency = currency;
     this.amount = Math.round(amount * centFactor());
  }
  public Money(long amount, Currency currency) {
     this.currency = currency;
     this.amount = amount * centFactor();
  }
  private static final int[] cents = new int[] { 1, 10, 100, 1000 };
  private int centFactor() {
     return cents[currency.getDefaultFractionDigits()];
  }
```


From the Library of Kyle Geoffrey Passarelli

Different currencies have different fractional amounts. The Java 1.4 Currency class will tell you the number of fractional digits in a class. We can determine how many minor units there are in a major unit by raising ten to the power, but that's such a pain in Java that the array is easier (and probably quicker). We're prepared to live with the fact that this code breaks if someone uses four fractional digits.

Although most of the time you'll want to use money operation directly, there are occasions when you'll need access to the underlying data.

```
class Money...

   public BigDecimal amount() {
      return BigDecimal.valueOf(amount, currency.getDefaultFractionDigits());
   }
   public Currency currency() {
      return currency;
   }
```

You should always question your use of accessors. There's almost always a better way that won't break encapsulation. One example that we couldn't avoid is database mapping, as in *Embedded Value (268)*.

If you use one currency very frequently for literal amounts, a helper constructor can be useful.

```
class Money...

   public static Money dollars(double amount) {
      return new Money(amount, Currency.USD);
   }
```

As *Money* is a *Value Object (486)* you'll need to define equals.

```
class Money...

   public boolean equals(Object other) {
      return (other instanceof Money) && equals((Money)other);
   }
   public boolean equals(Money other) {
      return currency.equals(other.currency) && (amount == other.amount);
   }
```


And wherever there's an equals there should be a hash.

```
class Money...

   public int hashCode() {
      return (int) (amount ^ (amount >>> 32));
   }
```

From the Library of Kyle Geoffrey Passarelli

We'll start going through the arithmetic with addition and subtraction.

```
class Money...

   public Money add(Money other) {
      assertSameCurrencyAs(other);
      return newMoney(amount + other.amount);
   }
   private void assertSameCurrencyAs(Money arg) {
      Assert.equals("money math mismatch", currency, arg.currency);
   }
   private Money newMoney(long amount) {
      Money money = new Money();
      money.currency = this.currency;
      money.amount = amount;
      return money;
   }
```

Note the use of a private factory method here that doesn't do the usual conversion into the cent-based amount. We'll use that a few times inside the *Money* code itself.

With addition defined, subtraction is easy.

```
class Money...

   public Money subtract(Money other) {
      assertSameCurrencyAs(other);
      return newMoney(amount - other.amount);
   }
```

The base method for comparison is `compareTo`.

```
class Money...

   public int compareTo(Object other) {
      return compareTo((Money)other);
   }
   public int compareTo(Money other) {
      assertSameCurrencyAs(other);
      if (amount < other.amount) return -1;
      else if (amount == other.amount) return 0;
      else return 1;
   }
```

Although that's all you get on most Java classes these days, we find code is more readable with the other comparison methods such as these.

```
class Money...

   public boolean greaterThan(Money other) {
      return (compareTo(other) > 0);
   }
```


From the Library of Kyle Geoffrey Passarelli

Now we're ready to look at multiplication. We're providing a default rounding mode but you can set one yourself as well.

```
class Money...

   public Money multiply(double amount) {
      return multiply(new BigDecimal(amount));
   }
   public Money multiply(BigDecimal amount) {
      return multiply(amount, BigDecimal.ROUND_HALF_EVEN);
   }
   public Money multiply(BigDecimal amount, int roundingMode) {
      return new Money(amount().multiply(amount), currency, roundingMode);
   }
```

If you want to allocate a sum of money among many targets and you don't want to lose cents, you'll want an allocation method. The simplest one allocates the same amount (almost) amongst a number of targets.

```
class Money...

   public Money[] allocate(int n) {
      Money lowResult = newMoney(amount / n);
      Money highResult = newMoney(lowResult.amount + 1);
      Money[] results = new Money[n];
      int remainder = (int) amount % n;
      for (int i = 0; i < remainder; i++) results[i] = highResult;
      for (int i = remainder; i < n; i++) results[i] = lowResult;
      return results;
   }
```

A more sophisticated allocation algorithm can handle any ratio.

```
class Money...

   public Money[] allocate(long[] ratios) {
      long total = 0;
      for (int i = 0; i < ratios.length; i++) total += ratios[i];
      long remainder = amount;
      Money[] results = new Money[ratios.length];
      for (int i = 0; i < results.length; i++) {
         results[i] = newMoney(amount * ratios[i] / total);
         remainder -= results[i].amount;
      }
      for (int i = 0; i < remainder; i++) {
         results[i].amount++;
      }
      return results;
   }
```


From the Library of Kyle Geoffrey Passarelli

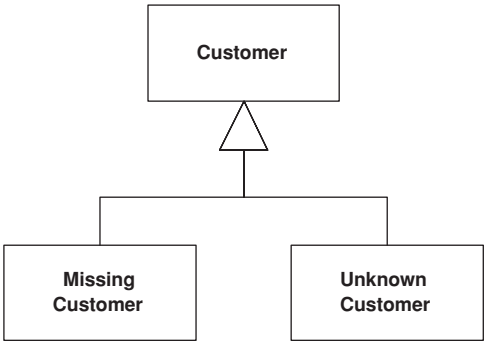
You can use this to solve Foemmel's Conundrum.

```
class Money...

   public void testAllocate2() {
      long[] allocation = {3,7};
      Money[] result = Money.dollars(0.05).allocate(allocation);
      assertEquals(Money.dollars(0.02), result[0]);
      assertEquals(Money.dollars(0.03), result[1]);
   }
```


From the Library of Kyle Geoffrey Passarelli

Special Case

A subclass that provides special behavior for particular cases.



Nulls are awkward things in object-oriented programs because they defeat polymorphism. Usually you can invoke `foo` freely on a variable reference of a given type without worrying about whether the item is the exact type or a subclass. With a strongly typed language you can even have the compiler check that the call is correct. However, since a variable can contain null, you may run into a runtime error by invoking a message on null, which will get you a nice, friendly stack trace.

If it's possible for a variable to be null, you have to remember to surround it with null test code so you'll do the right thing if a null is present. Often the right thing is the same in many contexts, so you end up writing similar code in lots of places—committing the sin of code duplication.

Nulls are a common example of such problems and others crop up regularly. In number systems you have to deal with infinity, which has special rules for things like addition that break the usual invariants of real numbers. One of my earliest experiences in business software was with a utility customer who wasn't fully known, referred to as "occupant." All of these imply altering the usual behavior of the type.

Instead of returning null, or some odd value, return a *Special Case* that has the same interface as what the caller expects.

From the Library of Kyle Geoffrey Passarelli

How It Works

The basic idea is to create a subclass to handle the *Special Case*. Thus, if you have a customer object and you want to avoid null checks, you make a null customer object. Take all of the methods for customer and override them in the *Special Case* to provide some harmless behavior. Then, whenever you have a null, put in an instance of null customer instead.

There's usually no reason to distinguish between different instances of null customer, so you can often implement a *Special Case* with a flyweight [Gang of Four]. You can't do it all the time. For a utility you can accumulate charges against an occupant customer even you can't do much billing, so it's important to keep your occupants separate.

A null can mean different things. A null customer may mean no customer or it may mean that there's a customer but we don't know who it is. Rather than just using a null customer, consider having separate *Special Cases* for missing customer and unknown customer.

A common way for a *Special Case* to override methods is to return another *Special Case*, so if you ask an unknown customer for his last bill, you may well get an unknown bill.

IEEE 754 floating-point arithmetic offers good examples of *Special Case* with positive infinity, negative infinity, and not-a-number (NaN). If you divide by zero, instead of getting an exception that you have to deal with, the system just returns NaN, and NaN participates in arithmetic just like any other floating point number.

When to Use It

Use *Special Case* whenever you have multiple places in the system that have the same behavior after a conditional check for a particular class instance, or the same behavior after a null check.

Further Reading

I haven't seen *Special Case* written up as a pattern yet, but **Null Object** has been written up in [Woolf]. If you'll pardon the unresistable pun, I see Null Object as special case of *Special Case*.

From the Library of Kyle Geoffrey Passarelli

Example: A Simple Null Object (C#)

Here's a simple example of *Special Case* used as a null object.

We have a regular employee.

```
class Employee...

      public virtual String Name {
         get {return _name;}
         set {_name = value;}
      }
      private String _name;
      public virtual Decimal GrossToDate {
         get {return calculateGrossFromPeriod(0);}
      }
      public virtual Contract Contract {
         get {return _contract;}
      }
      private Contract _contract;
```

The features of the class could be overridden by a null employee

```
class NullEmployee : Employee, INull...

      public override String Name {
         get {return "Null Employee";}
         set {}
      }
      public override Decimal GrossToDate {
         get {return 0m;}
      }
      public override Contract Contract {
         get {return Contract.NULL;}
      }
```

Notice that when you ask a null employee for its contract you get a null contract back.

The default values here avoid a lot of null tests if they end up with the same null values. The repeated null values are handled by the null object by default. You can also test for nullness explicitly either by giving the customer an `isNull` method or by using a type test for a marker interface.

From the Library of Kyle Geoffrey Passarelli

Plugin

by David Rice and Matt Foemmel

Links classes during configuration rather than compilation.

Separated Interface (476) is often used when application code runs in multiple runtime environments, each requiring different implementations of particular behavior. Most developers supply the correct implementation by writing a factory method. Suppose you define your primary key generator with a *Separated Interface (476)* so that you can use a simple in-memory counter for unit testing but a database-managed sequence for production. Your factory method will most likely contain a conditional statement that looks at a local environment variable, determines if the system is in test mode, and returns the correct key generator. Once you have a few factories you have a mess on your hands. Establishing a new deployment configuration—say "execute unit tests against in-memory database without transaction control" or "execute in production mode against DB2 database with full transaction control"—requires editing conditional statements in a number of factories, rebuilding, and redeploying. Configuration shouldn't be scattered throughout your application, nor should it require a rebuild or redeployment. *Plugin* solves both problems by providing centralized, runtime configuration.

How It Works

The first thing to do is define with a *Separated Interface (476)* any behaviors that will have different implementations based on runtime environment. Beyond that, we use the basic factory pattern, only with a few special requirements. The *Plugin* factory requires its linking instructions to be stated at a single, external point in order that configuration can be easily managed. Additionally, the linking to implementations must occur dynamically at runtime rather than during compilation, so that reconfiguration won't require a rebuild.

From the Library of Kyle Geoffrey Passarelli

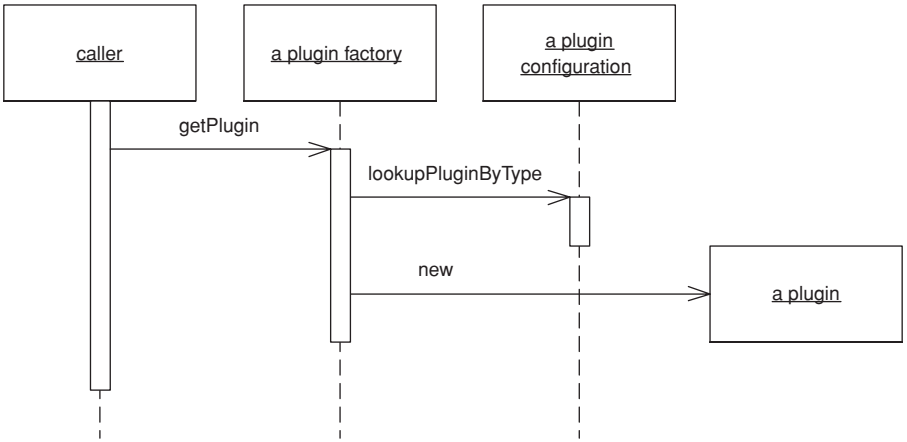


Figure 18.2 *A caller obtains a Plugin implementation of a separated interface.*

A text file works quite well as the means of stating linking rules. The *Plugin* factory will simply read the text file, look for an entry specifying the implementation of a requested interface, and return that implementation.

Plugin works best in a language that supports reflection because the factory can construct implementations without compile-time dependencies on them. When using reflection, the configuration file must contain mappings of interface names to implementation class names. The factory can sit independently in a framework package and needn't be changed when you add new implementations to your configuration options.

Even when not using a language that supports reflection it's still worthwhile to establish a central point of configuration. You can even use a text file to set up linking rules, with the only difference that your factory will use conditional logic to map an interface to the desired implementation. Each implementation type must be accounted for in the factory—not a big a deal in practice. Just add another option within the factory method whenever you add a new implementation to the code base. To enforce layer and package dependencies with a build-time check, place this factory in its own package to avoid breaking your build process.

When to Use It

Use *Plugin* whenever you have behaviors that require different implementations based on runtime environment.

From the Library of Kyle Geoffrey Passarelli

Example: An Id Generator (Java)

As discussed above, key, or ID, generation is a task whose implementation might vary between deployment environments (Figure 18.3).

First we'll write the IdGenerator *Separated Interface (476)* as well as any needed implementations.

```
interface IdGenerator...

   public Long nextId();

class OracleIdGenerator implements IdGenerator...

   public OracleIdGenerator() {
      this.sequence = Environment.getProperty("id.sequence");
      this.datasource = Environment.getProperty("id.source");
   }
```

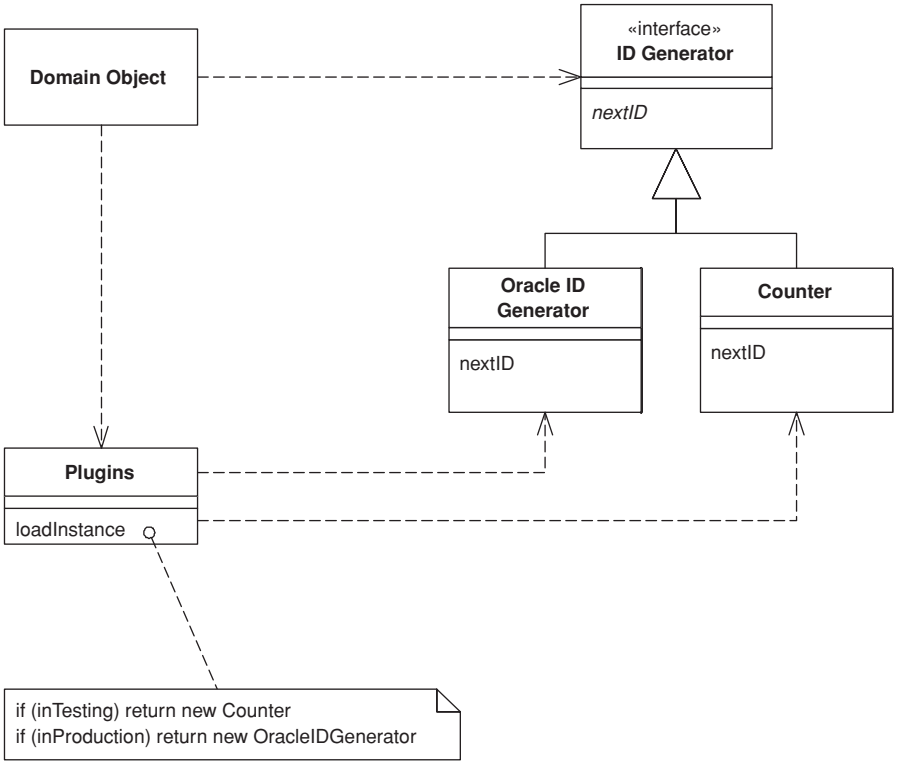


Figure 18.3 *Multiple ID generators.*

From the Library of Kyle Geoffrey Passarelli

In the OracleIdGenerator, nextId() selects the next available number out of the defined sequence from the defined data source.

```
class Counter implements IdGenerator...

   private long count = 0;
   public synchronized Long nextId() {
      return new Long(count++);
   }
```

Now that we have something to construct, let's write the plugin factory that will realize the current interface-to-implementation mappings.

```
class PluginFactory...
   private static Properties props = new Properties();

   static {
      try {
         String propsFile = System.getProperty("plugins");
         props.load(new FileInputStream(propsFile));
      } catch (Exception ex) {
         throw new ExceptionInInitializerError(ex);
      }
   }

   public static Object getPlugin(Class iface) {

      String implName = props.getProperty(iface.getName());
      if (implName == null) {
         throw new RuntimeException("implementation not specified for " +
                                    iface.getName() + " in PluginFactory propeties.");
      }
      try {
         return Class.forName(implName).newInstance();
      } catch (Exception ex) {
         throw new RuntimeException("factory unable to construct instance of " +
                                    iface.getName());
      }
   }
```

Note that we're loading the configuration by looking for a system property named plugins that will locate the file containing our linking instructions. Many options exist for defining and storing linking instructions, but we find a simple properties file the easiest. Using the system property to find the file rather than looking on the classpath makes it simple to specify a new configuration anywhere on your machine. This can be very convenient when moving builds between development, test, and production environments. Here's how two different configuration files, one for test and one for production, might look:

From the Library of Kyle Geoffrey Passarelli

```
config file test.properties...

   # test configuration
   IdGenerator=TestIdGenerator

config file prod.properties...

   # production configuration
   IdGenerator=OracleIdGenerator
```

Let's go back to the IdGenerator interface and add a static INSTANCE member that's set by a call to the *Plugin* factory. It combines *Plugin* with the singleton pattern to provide an extremely simple, readable call to obtain an ID.

```
interface IdGenerator...

   public static final IdGenerator INSTANCE =
      (IdGenerator) PluginFactory.getPlugin(IdGenerator.class);
```

We can now make that call knowing that we'll get the right ID for the right environment.

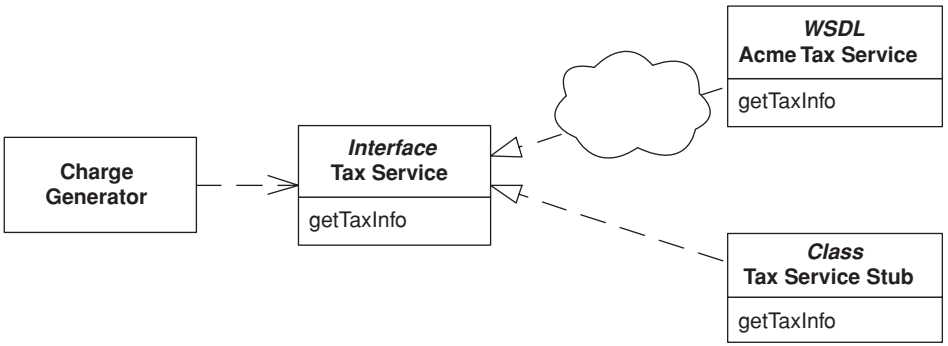
```
class Customer extends DomainObject...

   private Customer(String name, Long id) {
      super(id);
      this.name = name;
   }
   public Customer create(String name) {
      Long newObjId = IdGenerator.INSTANCE.nextId();
      Customer obj = new Customer(name, newObjId);
      obj.markNew();
      return obj;
   }
```


From the Library of Kyle Geoffrey Passarelli

Service Stub

by David Rice

Removes dependence upon problematic services during testing.



Enterprise systems often depend on access to third-party services such as credit scoring, tax rate lookups, and pricing engines. Any developer who has built such a system can speak to the frustration of being dependent on resources completely out of his control. Feature delivery is unpredictable, and as these services are often remote reliability and performance can suffer as well.

At the very least these problems slow the development process. Developers sit around waiting for the service to come back on line or maybe put some hacks into the code to compensate for yet to be delivered features. Much worse, and quite likely, such dependencies will lead to times when tests can't execute. When tests can't run the development process is broken.

Replacing the service during testing with a *Service Stub* that runs locally, fast, and in memory improves your development experience.

How It Works

The first step is to define access to the service with a *Gateway (466)*. The *Gateway (466)* should not be a class but rather a *Separated Interface (476)* so you can have one implementation that calls the real service and at least one that's only a *Service Stub*. The desired implementation of the *Gateway (466)* should

From the Library of Kyle Geoffrey Passarelli

be loaded using a *Plugin (499)*. The key to writing a *Service Stub* is that you keep it as simple as possible—complexity will defeat your purpose.

Let's walk through the process of stubbing a sales tax service that provides state sales tax amount and rate, given an address, product type, and sales amount. The simplest way to provide a *Service Stub* is to write two or three lines of code that use a flat tax rate to satisfy all requests.

Tax laws aren't that simple, of course. Certain products are exempt from taxation in certain states, so we rely on our real tax service to know which product and state combinations are tax exempt. However, a lot of our application functionality depends on whether taxes are charged, so we need to accommodate tax exemption in our *Service Stub*. The simplest means of adding this behavior to the stub is via a conditional statement that exempt a specific combination of address and product and then uses that same data in any relevant test cases. The number of lines of code in our stub can still be counted on one hand.

A more dynamic *Service Stub* maintains a list of exempt product and state combinations, allowing test cases to add to it. Even here we're at about 10 lines of code. We're keeping things simple given our aim of speeding the development process.

The dynamic *Service Stub* brings up an interesting question regarding the dependency between it and test cases. The *Service Stub* relies on a setup method for adding exemptions that isn't in the original tax service *Gateway (466)* interface. To take advantage of a *Plugin (499)* to load the *Service Stub,* this method must be added to the *Gateway (466),* which is fine as it doesn't add much noise to your code and is done in the name of testing. Be sure that the *Gateway (466)* implementation that calls the real service throws assertion failures within any test methods.

When to Use It

Use *Service Stub* whenever you find that dependence on a particular service is hindering your development and testing.

Many practitioners of Extreme Programming use the term **Mock Object**, for a *Service Stub*. We've stuck with *Service Stub* because it's been around longer.

Example: Sales Tax Service (Java)

Our application uses a tax service that's deployed as a Web service. The first item we'll take care of is defining a *Gateway (466)* so that our domain code isn't

From the Library of Kyle Geoffrey Passarelli

forced to deal with the wonders of Web services. The *Gateway (466)* is defined as an interface to facilitate loading of any *Service Stubs* that we write. We'll use *Plugin (499)* to load the correct tax service implementation.

```
interface TaxService...

   public static final TaxService INSTANCE =
      (TaxService) PluginFactory.getPlugin(TaxService.class);
   public TaxInfo getSalesTaxInfo(String productCode, Address addr, Money saleAmount);
```

The simple flat rate *Service Stub* would look like this:

```
class FlatRateTaxService implements TaxService...

   private static final BigDecimal FLAT_RATE = new BigDecimal("0.0500");
   public TaxInfo getSalesTaxInfo(String productCode, Address addr, Money saleAmount) {
      return new TaxInfo(FLAT_RATE, saleAmount.multiply(FLAT_RATE));
   }
```

Here's a *Service Stub* that provides tax exemptions for a particular address and product combination:

```
class ExemptProductTaxService implements TaxService...

   private static final BigDecimal EXEMPT_RATE = new BigDecimal("0.0000");
   private static final BigDecimal FLAT_RATE = new BigDecimal("0.0500");
   private static final String EXEMPT_STATE = "IL";
   private static final String EXEMPT_PRODUCT = "12300";
   public TaxInfo getSalesTaxInfo(String productCode, Address addr, Money saleAmount) {
      if (productCode.equals(EXEMPT_PRODUCT) && addr.getStateCode().equals(EXEMPT_STATE)) {
         return new TaxInfo(EXEMPT_RATE, saleAmount.multiply(EXEMPT_RATE));
      } else {
         return new TaxInfo(FLAT_RATE, saleAmount.multiply(FLAT_RATE));
      }
   }
```

Now here's a more dynamic *Service Stub* whose methods allow a test case to add and reset exemption combinations. Once we feel it necessary to add these test methods we need to go back and add these methods to our simpler *Service Stubs* as well as to the implementation that calls the actual tax Web service. The unused test methods should all throw assertion failures.


```
class TestTaxService implements TaxService...

   private static Set exemptions = new HashSet();
   public TaxInfo getSalesTaxInfo(String productCode, Address addr, Money saleAmount) {
      BigDecimal rate = getRate(productCode, addr);
      return new TaxInfo(rate, saleAmount.multiply(rate));
   }
   public static void addExemption(String productCode, String stateCode) {
      exemptions.add(getExemptionKey(productCode, stateCode));
   }
```

From the Library of Kyle Geoffrey Passarelli

```
public static void reset() {
   exemptions.clear();
}
private static BigDecimal getRate(String productCode, Address addr) {
   if (exemptions.contains(getExemptionKey(productCode, addr.getStateCode()))) {
      return EXEMPT_RATE;
   } else {
      return FLAT_RATE;
   }
}
```

Not shown is the implementation that calls the Web service providing our real tax data, to which our production *Plugin (499)* configuration would link the tax service interface. Our test *Plugin (499)* configurations would link to the appropriate *Service Stub* above.

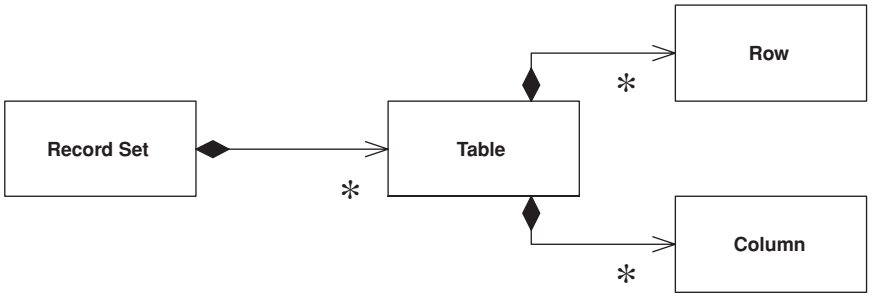
Finally, any caller to the tax service must access the service via the *Gateway (466)*. We have a charge generator here that creates standard charges and then calls the tax service in order to create any corresponding taxes.

```
class ChargeGenerator...

   public Charge[] calculateCharges(BillingSchedule schedule) {
      List charges = new ArrayList();
      Charge baseCharge = new Charge(schedule.getBillingAmount(), false);
      charges.add(baseCharge);
      TaxInfo info = TaxService.INSTANCE.getSalesTaxInfo(
            schedule.getProduct(), schedule.getAddress(), schedule.getBillingAmount());
      if (info.getStateRate().compareTo(new BigDecimal(0)) > 0) {
         Charge taxCharge = new Charge(info.getStateAmount(), true);
         charges.add(taxCharge);
      }
      return (Charge[]) charges.toArray(new Charge[charges.size()]);
   }
```


From the Library of Kyle Geoffrey Passarelli

Record Set

An in-memory representation of tabular data.



In the last twenty years, the dominant way to represent data in a database has been the tabular relational form. Backed by database companies big and small, and a fairly standard query language, almost every new development I see uses relational data.

On top of this has come a wealth of tools for building UI's quickly. These data-aware UI frameworks rely on the fact that the underlying data is relational, and they provide UI widgets of various kinds that make it easy to view and manipulate this data with almost no programming.

The dark side of these environments is that, while they make display and simple updates ridiculously easy, they have no real facilities in which to place business logic. Any validations beyond "is this a valid date," and any business rules or computations have no good place to go. Either they're jammed into the database as stored procedures or they're mingled with UI code.

The idea of the *Record Set* is to give you your cake and let you eat it, by providing an in-memory structure that looks exactly like the result of an SQL query but can be generated and manipulated by other parts of the system.

How It Works

A *Record Set* is usually something that you won't build yourself, provided by the vendor of the software platform you're working with. Examples include the data set of ADO.NET and the row set of JDBC 2.0.

The first essential element of a *Record Set* is that it looks exactly like the result of a database query. That means you can use the classical two-tier

From the Library of Kyle Geoffrey Passarelli

approach of issuing a query and throwing the data directly into a data-aware UI with all the ease that these two-tier tools give you. The second essential element is that you can easily build a *Record Set* yourself or take one that has resulted from a database query and easily manipulate it with domain logic code.

Although platforms often give you a *Record Set,* you can create one yourself. The problem is that there isn't that much point without the data-aware UI tools, which you would also need to create yourself. In any case it's fair to say that building a *Record Set* structure as a list of maps, which is common in dynamically typed scripting languages, is a good example of this pattern.

The ability to disconnect the *Record Set* from its link to the data source is very valuable. This allows you to pass the *Record Set* around a network without having to worry about database connections. Furthermore, if you can then easily serialize the *Record Set* it can also act as a *Data Transfer Object (401)* for an application.

Disconnection raises the question of what happens when you update the *Record Set*. Increasingly platforms are allowing the *Record Set* to be a form of *Unit of Work (184)*, so you can modify it and then return it to the data source to be committed. A data source can typically use *Optimistic Offline Lock (416)* to see if there are any conflicts and, if not, write the changes out to the database.

Explicit Interface Most *Record Set* implementations use an **implicit interface**. This means that to get information out of the *Record Set* you invoke a generic method with an argument to indicate which field you want. For example, to get the passenger on an airline reservation you use an expression like `aReservation["passenger"]`. An explicit interface requires a real reservation class with defined methods and properties. With an explicit reservation the expression for a passenger might be `aReservation.passenger`.

Implicit interfaces are flexible in that you can use a generic *Record Set* for any kind of data. This saves you having to write a new class every time you define a new kind of *Record Set*. In general, however, I find implicit interfaces to be a Bad Thing. If I'm programming with a reservation, how do I know how to get the passenger? Is the appropriate string "passenger" or "guest" or "flyer"? The only way I can tell is to wander around the code base trying to find where reservations are created and used. If I have an explicit interface I can look at the definition of the reservation to see what property I need.

This problem is exacerbated with statically typed languages. If I want the last name of the passenger, I have to resort to some horrible expression such as `((Person)aReservation["passenger"]).lastName`, but then the compiler loses all type information and I have to manually enter it in to get the information I want. An explicit interface can keep the type information so I can use `aReservation.passenger.lastName`.

From the Library of Kyle Geoffrey Passarelli

For these reasons, I generally frown on implicit interfaces (and their evil cousin, passing data around in dictionaries). I'm also not too keen on them with *Record Sets*, but the saving grace here is that the *Record Set* usually carries information on the legal columns in it. Furthermore, the column names are defined by the SQL that creates the *Record Set*, so it's not too difficult to find the properties when you need them.

But it's nice to go further and have an explicit interface. ADO.NET provides this with its strongly typed data sets, generated classes that provide an explicit and fully typed interface for a *Record Set*. Since an ADO.NET data set can contain many tables and the relationships between them, strongly typed data sets also provide properties that can use that relationship information. The classes are generated from the XSD data set definition.

Implicit interfaces are more common, so I've used untyped data sets in my examples for this book. For production code in ADO.NET, however, I suggest using data sets that are typed. In a non-ADO.NET environment, I suggest using code generation for your own explicit *Record Sets*.

When to Use It

To my mind the value of *Record Set* comes from having an environment that relies on it as a common way of manipulating data. A lot of UI tools use *Record Set*, and that's a compelling reason to use them yourself. If you have such an environment, you should use *Table Module (125)* to organize your domain logic: Get a *Record Set* from the database; pass it to a *Table Module (125)* to calculate derived information; pass it to a UI for display and editing; and pass it back to a *Table Module (125)* for validation. Then commit the updates to the database.

In many ways the tools that make *Record Set* so valuable appeared because of the ever-presence of relational databases and SQL and the absence of any real alternative structure and query language. Now, of course, there's XML, which has a widely standardized structure and a query language in XPath, and I think it's likely that we'll see tools that use a hierarchic structure in the same way that current tools now use *Record Set*. Perhaps this is actually a particular case of a more generic pattern: something like *Generic Data Structure*. But I'll leave thinking about that pattern until then.

From the Library of Kyle Geoffrey Passarelli